Anexo Técnico de Seguridad

Autor: Adrián López Delgado

Curso: 2º DAM (Desarrollo de Aplicaciones Multiplataforma)

Centro: FP Nebrija Madrid

Módulo: Desarrollo de Interfaces

Fecha: Octubre 2026

---

1. Enfoque General de Seguridad

Cuando me planteé el diseño de esta aplicación para la gestión de inventario y activos hardware de una empresa, tuve muy claro que la seguridad no podía ser un añadido de última hora, sino una parte fundamental de la arquitectura. En un entorno corporativo real, los datos sobre servidores, conmutadores de red o cortafuegos son información sensible: contienen precios de compra, cantidades disponibles en almacén, nombres de responsables técnicos y registros de auditoría que deben protegerse tanto de accesos no autorizados como de manipulaciones indebidas.

Para lograrlo, he aplicado el **principio del menor privilegio** (cada usuario solo puede ver y hacer lo estrictamente necesario para su trabajo) y una estrategia de **defensa en profundidad**, protegiendo la aplicación en múltiples niveles: en las rutas del navegador, en el manejo de sesiones mediante tokens, en el tráfico de peticiones mediante interceptores, en la validación estricta de formularios y en la prevención de ataques al exportar archivos.

A continuación, explico con detalle cómo he implementado cada una de estas capas.

---

2. Control de Acceso Basado en Roles (RBAC)

Para gestionar los permisos he implementado un modelo de control de acceso basado en roles (Role-Based Access Control). En lugar de conceder permisos de forma caótica usuario por usuario, el sistema define perfiles claros con un alcance determinado:

2.1. Matriz de Roles y Privilegios

Función / Módulo del Sistema	Rol: Administrador (`ADMIN`)	Rol: Operador Estándar (`STANDARD`)	Usuario No Autenticado (Público)
Portal de Acceso (Login)	Permitido	Permitido	Permitido
Manual de Usuario y Ayuda (`/ayuda`)	Permitido	Permitido	Permitido
Cuadro de Mando Ejecutivo (`/dashboard`)	Permitido (Acceso Total)	Denegado (Redirige a Error 403)	Denegado (Redirige a Login)
Gráfico Interactivo de Métricas (Chart.js)	Permitido (Lectura y filtros)	Denegado	Denegado
Bitácora de Auditoría NoSQL en vivo	Permitido (Supervisión)	Denegado	Denegado
Exportación Oficial (PDF y CSV)	Permitido (Descarga)	Denegado	Denegado

Función / Módulo del Sistema	Rol: Administrador (`ADMIN`)	Rol: Operador Estándar (`STANDARD`)	Usuario No Autenticado (Público)
Formulario de Alta de Hardware (`/registros`)	Permitido (Creación y edición)	Permitido (Creación y edición)	Denegado (Redirige a Login)
Consulta de Catálogo de Inventario	Permitido (Búsqueda y ordenación)	Permitido (Búsqueda y ordenación)	Denegado (Redirige a Login)
Eliminación de Equipos del Catálogo	Permitido	Permitido (Requiere confirmación)	Denegado

2.2. Por qué separar estos roles

El perfil de Administrador está concebido para jefes de departamento o directores de sistemas que necesitan supervisar costes financieros globales, analizar tendencias de inventario y auditar qué persona ha realizado cada modificación.

Por otro lado, los operadores técnicos necesitan agilidad para registrar nuevas entradas de material o consultar si queda stock de un conmutador en almacén, pero no deben tener visibilidad sobre métricas de balance económico consolidado ni acceso directo a la bitácora interna de seguridad.

---

3. Protección de Rutas en el Cliente (Guards de Angular)

En una aplicación de una sola página (SPA), el usuario puede intentar cambiar la URL del navegador manualmente escribiendo por ejemplo `/dashboard`. Para impedir que alguien acceda a vistas restringidas saltándose la interfaz, he programado dos barreras de seguridad denominadas **Guards**:

3.1. Guardia de Autenticación (`authGuard`)

Su cometido es muy sencillo pero vital: antes de que el enrutador de Angular cargue cualquier pantalla privada, comprueba si existe una sesión activa y válida en el servicio `AuthService`.

- Si el usuario está correctamente identificado, le deja pasar.
- Si el usuario no ha iniciado sesión, detiene la navegación en seco, guarda la dirección a la que intentaba ir mediante el parámetro `returnUrl` y le envía a la pantalla de `/login`. De esta forma, en cuanto inicie sesión con éxito, volverá directamente a la pantalla que estaba buscando sin perder tiempo.

3.2. Guardia de Roles (`roleGuard`)

Esta guardia protege específicamente las rutas que requieren privilegios elevados, como el Cuadro de Mando Ejecutivo (`/dashboard`).

- He programado esta comprobación como una función pura moderna (`CanActivateFn`) que recibe el rol mínimo requerido.
- Si un usuario que ha iniciado sesión como `STANDARD` intenta entrar a `/dashboard`, la guardia detecta que su rol no cumple la condición, bloquea la carga del componente y redirige al usuario a una pantalla específica de **Acceso No Autorizado (Error 403 Forbidden)** en la ruta `/unauthorized`.

3.3. Tratamiento de la pantalla de error 403

En la pantalla de acceso denegado no expongo ningún detalle sensible ni rutas internas del servidor. Únicamente informo al usuario de que su rol actual no cuenta con privilegios suficientes y le ofrezco dos opciones seguras: cerrar sesión para entrar con una cuenta con mayores privilegios o volver al manual de usuario para consultar la matriz de permisos.

---

4. Gestión de Sesiones, Tokens JWT e Interceptor HTTP

La sesión del usuario no se gestiona mediante simples variables en memoria que se pierden al recargar la página, ni mediante cookies vulnerables a ataques CSRF. He implementado un sistema basado en **Tokens de Acceso (JWT)**:

4.1. Generación y almacenamiento del token

Cuando el usuario introduce sus credenciales correctamente en el login:

1. El servicio valida la contraseña comprobando su firma contra la base de datos simulada.

2. Genera un token estructurado en formato estándar JWT compuesto por tres partes separadas por puntos:

- **Cabecera (Header):** Indica el algoritmo criptográfico (`HS256`) y el tipo de token (`JWT`).
- **Carga útil (Payload):** Codifica en Base64 la información esencial de la sesión (identificador de usuario, correo, nombre, rol asignado y fecha de emisión).
- **Firma (Signature):** Simulación de firma digital con clave secreta corporativa para evitar que el contenido del token sea manipulado en el cliente.

3. El token y los datos de la sesión se almacenan de forma controlada en el almacenamiento local del navegador (`localStorage`), permitiendo mantener la sesión activa si el usuario recarga la ventana.

4.2. Interceptor HTTP de Seguridad (`authInterceptor`)

Para garantizar que toda comunicación saliente sea segura y esté autenticada, he programado un interceptor funcional que captura automáticamente cada petición HTTP que realiza la aplicación antes de que salga a la red:

- **Inyección de cabecera Bearer:** Si existe un token activo en el sistema, el interceptor añade automáticamente la cabecera estándar:

`Authorization: Bearer`

además de cabeceras de identificación de cliente (`X-Requested-With` y `X-App-Version`). De este modo no es necesario añadir el token a mano en cada servicio de datos.

- **Captura reactiva de errores de seguridad:**
- Si el servidor simulado responde con un código **401 Unauthorized** (lo que significa que el token ha caducado o es ilegítimo), el interceptor borra de inmediato la sesión almacenada en el navegador para evitar suplantaciones y redirige al usuario a la pantalla de login con el aviso de sesión expirada.
- Si responde con un código **403 Forbidden**, redirige directamente a la pantalla de acceso denegado.

---

5. Validación y Sanitización de Datos en Formularios

Un punto crítico de cualquier aplicación informática son las entradas de datos que teclea el usuario. Si no se validan adecuadamente, pueden provocar errores de cálculo, registros duplicados o fallos en el sistema.

En el formulario de alta de hardware he configurado una doble barrera de validación:

5.1. Validaciones síncronas estrictas

Se comprueban en tiempo real en la propia interfaz:

- **Obligatoriedad:** Ningún campo puede quedar en blanco.
- **Longitudes mínimas:** El nombre del equipo debe tener al menos 3 caracteres para evitar registros sin sentido como "a" o "x".
- **Rangos numéricos:** El stock no admite números negativos (`Validators.min(0)`) y el precio por unidad debe ser estrictamente positivo (`Validators.min(1)`).

- **Formato de correo electrónico:** Se verifica que el correo del técnico responsable cumpla con la estructura formal estándar (`usuario@dominio.com`).

5.2. Validaciones asíncronas con Debounce (Prevención de colisiones y saturación)

El código SKU es el identificador único de cada máquina (su matrícula dentro de la empresa). Dos servidores distintos no pueden compartir jamás el mismo SKU.

Para verificar que un SKU o un correo no estén ya registrados en la base de datos sin saturar el sistema con cientos de peticiones mientras el usuario teclea, he programado los validadores `uniqueSkuValidator` y `uniqueEmailValidator` :

- **Técnica de Debounce reactivo:** Cuando el usuario pulsa una tecla, el validador no consulta de inmediato a la base de datos; espera un lapso de 300 a 350 milisegundos (`timer(350)` con `switchMap` de RxJS). Si el usuario sigue escribiendo, la comprobación anterior se cancela y solo se lanza cuando el usuario deja de escribir.
- Mientras la comprobación está en marcha, el formulario muestra un indicador de estado pendiente (`PENDING`). Si el identificador ya existe, invalida el campo e impide que se active el botón de guardar.

---

6. Seguridad en la Exportación y Manejo de Archivos

Una vulnerabilidad muy común en aplicaciones web que exportan datos a hojas de cálculo es la **Inyección en CSV (Formula Injection)**. Ocurre cuando un usuario malintencionado introduce como nombre de un producto un texto que empieza por símbolos especiales como `=` , `+` , `-` o `@` . Si ese texto se vuelca directamente a un archivo CSV, cuando otro empleado abre el archivo en Microsoft Excel, el programa puede interpretar ese texto como una fórmula matemática o ejecutar comandos del sistema.

Para evitar este riesgo en mi servicio `ExportService` :

- **Escapado de caracteres y comillas:** Cada celda de texto que se va a incluir en el archivo CSV pasa por una función de sanitización que detecta si contiene caracteres de control, saltos de línea o comillas dobles, encerrando los textos entre comillas dobles debidamente escapadas (`""`).
- **Inyección de cabecera UTF-8 BOM:** Añado al inicio del archivo la marca de orden de bytes (`\uFEFF`). Esto le indica a programas como Microsoft Excel que el archivo utiliza codificación UTF-8 pura, impidiendo que el software intente adivinar la codificación y corrompa caracteres acentuados o el símbolo del euro.
- **Generación limpia de PDF:** El informe en PDF se ensambla íntegramente en la memoria del navegador utilizando la librería `jspdf` y `jspdf-autotable` , sin llamar a ningún ejecutable externo en el sistema del usuario ni cargar scripts remotos en tiempo de ejecución.

---

7. Políticas de Cabeceras HTTP y Despliegue Seguro

Para cuando la aplicación se despliegue en un servidor o plataforma en la nube (como Vercel o Netlify), he preparado un archivo de configuración de seguridad (`vercel.json`) que inyecta en cada respuesta del servidor las cabeceras de protección más recomendadas por la industria:

1. **Content Security Policy (CSP):**

- Controla de qué orígenes exactos está permitido cargar fuentes, estilos, scripts o imágenes, impidiendo la inyección de código malicioso mediante ataques XSS (Cross-Site Scripting).

2. **X-Frame-Options:** `DENY` **:**

- Impide que otras páginas web puedan incrustar nuestra aplicación dentro de un marco invisible (

